MANUAL TECNICO

LFP
2do semestre
David Castañeda
202307821

Introducción:

El siguiente programa tiene como finalidad hacer uso de la programación orientada a objetos y el manejo de archivos en javascript.

Dentro de este se podrán elegir 8 acciones para el manejo de archivos:

1. **Historial de Llamadas:** Deberá generar una tabla con los datos del operador, cliente y la calificación de la llamada.
2. **Listado de Operadores:** Deberá generar una tabla con los datos del operador (id y nombre).
3. **Listado de Clientes:** Deberá generar una tabla con los datos del operador (id y nombre).
4. **Rendimiento de Cada Operador:** Deberá generar una tabla con los datos del operador (id, nombre y porcentaje de atención).
    - El porcentaje de atención se calculará de la siguiente forma: $porcentaje\ de\ atención = \frac{cantidad\ de\ llamadas\ atendidas\ por\ el\ operador}{cantidad\ de\ llamadas\ recibidas\ globalmente} * 100$
5. **Porcentaje de Clasificación de Llamadas:** Deberá mostrar en consola el porcentaje de llamadas buenas, medias y malas.
6. **Cantidad de Llamadas por Calificación:** Deberá mostrar en consola la cantidad de llamadas de 1, 2, 3, 4 y 5 estrellas.

7. ingresar archivos para su uso en las anteriores elecciones.

8. salir del programa: para finalizar el programa cuando no necesitemos algo mas.

Fase 1 Inicializacion del programa:

Creamos variables que nos servirán para la lectura y escritura de archivos.

Path es útil para trabajar rutas dentro de javascript.

Fs nos ayuda a darle lectura a archivos

Con Readline podremos darle lectura a cada una de las líneas de un archivo.

Interface: verifica que la estructura de un objeto tenga los métodos que debe incluir.

```
const fs = require('fs');
const path = require('path');
const readline = require('readline');

const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout
  });
```

Class Menu:

La clase menú será la clase principal donde se juntaran cada una de las demás clases.

Constructor: dentro del constructor crearemos un arreglo llamado this.datos que almacenara los archivos que llamemos en el programa.

Start: la función start será la que dará inicio al programa, incluye un mensaje de inicio y esperara a que se muestre la siguiente función menú.

```
    async start(){
        console.log("Se inicio el programa");
        await this.menu();
```

Menú: es una función integrada a la clase menú, dentro de ella podremos visualizar las diferentes opciones que podremos ingresar, que toma valores en este caso numéricos y con esto se activaran diferentes secuencias y opciones para el funcionamiento del programa, así mismo dentro de esta espera a las funciones manejaropcion y preguntarInfo para seguir con el programa.

```
async menu(){
console.log('\n--------------- Menú Principal-------------------');
console.log('1. Cargar Registros de Llamadas');
console.log('2. Exportar Historial de Llamadas');
console.log('3. Exportar Listado de Operadores');
console.log('4. Exportar Listado de Clientes');
console.log('5. Exportar Rendimiento de Operadores');
console.log('6. Mostrar Porcentaje de Clasificación de Llamadas');
console.log('7. Mostrar Cantidad de Llamadas por Calificación');
console.log('8. Salir');
const opcion = await this.preguntarInfo("Elige una opcion: ");
await this.manejarOpcion(opcion);
}
```

preguntarInfo: en esta función se recibe un parámetro del usuario después de desplegar un mensaje solicitando un valor numérico que representa la opción que será evaluada en manejarOpcion, así mismo se creo una función similar para dar parámetros que serán utilizados mas adelante.

```
preguntarInfo(pregunta) {
  return new Promise((resultado) => {
    rl.question(pregunta, resultado);
  });
  }
preguntarInfo2(pregunta){
      return new Promise((resultado) =>{
          rl.question(pregunta, resultado);
      });
}
```

Manejaropcion:

Manejaropcion nos permite ingresar valores numéricos en este caso, para poder elegir las opciones que necesitemos seguir. Para ello hacemos uso de un switch para que tome el valor y lo compare en su lista para seguir con el programa, y con esto tener la opción que necesitemos.

```
async manejarOpcion(opcion){
  switch (opcion) {
    case '1':
      const opcion = await this.preguntarInfo2("Escriba el directorio del archivo: ");
      const extension = path.extname(opcion).toLowerCase();
      if(extension===".csv"){
        this.datos = cargarArchivos(opcion)
        console.log('Registros cargados correctamente.');
      } else{
        console.log("No es un archivo .csv");
      }
      this.menu();
      break;
    case '2':
      const exporta= new generarReportes();
      if(this.datos.length===0){
        console.log("no hay datos")
      } else{
        exporta.historial(this.datos);
        console.log(' Historial exportado como historial.html');
      }
      this.menu();
      break;
    case '3':
      const operadores = new exportarOperadores();
      if(this.datos.length===0){
        console.log("no hay datos");
      } else{
```

Fase 2 Opciones para manejo de archivos.

Carga de archivos .csv:

La primera opciones dentro del menú es la carga de un archivo .csv, en la que nos solicita directorio que se deberá ubicar en la misma carpeta para mas facilidades y esta evaluara si cumple con el requisito de terminar en ".csv".

Una vez se determine si cumple, esta hará un llamada a una función llamada cargaArchivos y asu vez estos serán almacenados dentro de un arreglo llamado this.datos que se encuentra dentro del constructor del menú.

función cargaArchivos:

dentro de esta función daremos lectura al archivo con el uso de la fs.readFileSync, en esta se especificara la ruta y comenzara a dar lectura a cada línea del archivo, estas serán almacenadas en un arreglo o array llamado registro para ser guardado en la memoria del equipo, dividiendo cada dato por cada "," gracias a Split.

También hacemos un llamado de atención al uso de una variable llamada estrellitas para contabilizar cada x como una estrella, esto nos será útil para las demás opciones del programa, especificando que las estrellas estarán divididas entre ";" y "," y un filter para buscar cada x dentro de estrellitas.

Luego de esto clasificara la contabilidad de cada estrella en 3 rangos, donde las estrellas sean 4 o mas serán de buena calidad, cuando sean iguales o mayores a 2 y

menores a 4 seran de calidad media, y las menores de 2 seran de calidad mala.

Por ultimo el return, permitirá devolver cada dato para su posterior análisis en las demás opciones del programa.

```
function cargarArchivos(rutaArchivo){
    const datos = fs.readFileSync(rutaArchivo,'utf-8');
    const linea = datos.trim().split('\n');

    return linea.slice(1).map(lineas =>{
      const valores = lineas.split(",");
      const registro = {
        id_operador: valores[0],
        nombre_operador: valores[1],
        id_cliente: valores[3],
        nombre_cliente: valores[4]
      }
      const estrellitas = lineas.split(/[;,]/).filter(estrellitas => estrellitas === 'x');
      let estrellas = estrellitas.length;
      let clasificacion = " ";
      if (estrellas>=4){
        clasificacion = 'Buena';
      } else if (estrellas>=2){
        clasificacion = 'Media';
      } else{
        clasificacion = 'Mala';
      }
      return {
      id_operador: registro.id_operador,
      nombre_operador: registro.nombre_operador,
      estrellas,
      clasificacion,
      id_cliente: registro.id_cliente,
      nombre_cliente: registro.nombre_cliente
    };
    })
}
```

Historial de llamadas:

Esta clase instanciada en el caso 2 de nuestro switch, permitirá hacer un historial de los datos dentro de nuestro archivo y pasarlos a una tabla en html, generando el mismo archivo durante el proceso.

```
class generarReportes{
    historial(registros) {
    let html = `<h1>Historial de Llamadas</h1><table border="1"><tr><th>Operador</th><th>Cliente</th><th>Estrellas</th></tr>`;
    registros.forEach(r => {
      html += `<tr><td>${r.nombre_operador}</td><td>${r.nombre_cliente}</td><td>${r.estrellas}</td></tr>`;
    });
    html += `</table>`;
    fs.writeFileSync('historial.html', html);
  }
}
```

exportarOperadores y ExportarClientes:

Ambas opciones 2 y 3 son similares en su funcionamiento, haciendo uso del archivo registro en this.datos, daremos un llamado a cada clase instanciada en una nueva variable para dar uso a las funciones exportar en el caso de exportarOperadores y clientes en el caso de seleccionar exportarClientes,

Cada clase hara historial de los datos dentro de nuestro archivo y pasarlos a una tabla en html, generando el mismo archivo durante el proceso gracias a fs.writeFileSync.

```
class exportarOperadores{
    exportar(registros){
        const operadores = [...new Map(registros.map(r => [r.id_operador, r.nombre_operador])).entries()];
        let html = `<h1>Listado de Operadores</h1><table border="1"><tr><th>ID</th><th>Nombre</th></tr>`;
        operadores.forEach(([id, nombre]) => {
            html += `<tr><td>${id}</td><td>${nombre}</td></tr>`;
        });
        html += `</table>`;
        fs.writeFileSync('operadores.html', html);
    }
}
class exportarClientes{
    clientes(rutaArchivo){
      const clientes = [...new Map(rutaArchivo.map(r => [r.id_cliente, r.nombre_cliente])).entries()];
      let html = `<h1>Listado de Clientes</h1><table border="1"><tr><th>ID</th><th>Nombre</th></tr>`;
      clientes.forEach(([id, nombre]) => {
      html += `<tr><td>${id}</td><td>${nombre}</td></tr>`;
      });
      html += `</table>`;
      fs.writeFileSync('clientes.html', html);
    }
}
```

```csv
id_operador,nombre_operador,estrellas,id_cliente,nombre_cliente
1,Henry,x;x;x;0;0,101,Enzo
2,Ferruccio,x;x;0;0;0,102,Luca
3,Maria,x;x;x;x;0,103,Sofía
1,Henry,x;0;0;0;0,104,Camila
4,Carlos,x;x;x;0;0,105,Diego
2,Ferruccio,x;x;x;0;0,106,Mateo
5,Ana,x;x;x;x;x,107,Isabella
3,Maria,x;x;0;0;0,108,Valentina
4,Carlos,x;0;0;0;0,109,Emiliano
2,Ferruccio,x;x;x;0;0,110,Sebastián
1,Henry,x;x;x;0;0,111,Gabriel
5,Ana,x;x;x;x;0,112,Martina
3,Maria,x;x;x;x;0,113,Lucas
2,Ferruccio,x;x;0;0;0,114,Juan
4,Carlos,x;x;x;x;x,115,Daniela
5,Ana,x;x;x;x;0,116,Alejandro
1,Henry,x;x;x;x;0,117,Julieta
3,Maria,x;0;0;0;0,118,Renata
2,Ferruccio,x;x;0;0;0,119,Tomás
5,Ana,x;x;x;x;0,120,Constanza
```

Rendimiento:

La clase rendimiento es similar a las 2 anteriores con el detalle de que aquí los datos son evaluados por medio de una operación matemática para evaluar el rendimiento de cada operador.

Se toma el valor de las cantidades tomadas por cada operador y se divide dentro de las llamadas recibidas de manera global, y al ser porcentaje se multiplica por 100 el resultado decimal.

```
class exportarRendimiento{
  rendimiento(rutaArchivo){
    const total = rutaArchivo.length;
      const operadores = {};
      rutaArchivo.forEach(r => {
        operadores[r.id_operador] = operadores[r.id_operador] || { nombre: r.nombre_operador, count: 0 };
        operadores[r.id_operador].count++;
      });
      let html = `<h1>Rendimiento de Operadores</h1><table border="1"><tr><th>ID</th><th>Nombre</th><th>% Atención</th></tr>`;
      for (const [id, data] of Object.entries(operadores)) {
        const porcentaje = ((data.count / total) * 100).toFixed(2);
        html += `<tr><td>${id}</td><td>${data.nombre}</td><td>${porcentaje}%</td></tr>`;
      }
      html += `</table>`;
      fs.writeFileSync('rendimiento.html', html)
  }
}
```

Porcentaje clasficacion de llamadas:

Esta clase evaluara las llamadas y mostrara en porcentajes el total de buenas, medias y malas.

Recorriendo la clasificación del generada del archivo y con un for iterar cada objeto, una vez obtenido el total de las calificaciones buenas, malas o medias, estas se dividirá por el total de llamadas usando un arreglo que mide la longitud de los datos y multiplicadas por 100 para obtener el porcentaje.

```
class clasificacion{
  mostrar(registros){
    const total = registros.length;
    const counts = { Buena: 0, Media: 0, Mala: 0 };
    registros.forEach(r => counts[r.clasificacion]++);
    console.log('\n Porcentaje de Clasificación:');
    for (const [tipo, count] of Object.entries(counts)) {
      console.log(`${tipo}: ${(count / total * 100).toFixed(2)}%`);
    }
  }
}
```

Cantidad de estrellas:

Esta clase, será la opción numero 7 y permitirá visualizar la cantidad de estrellas recibidas en la videollamada, desde 1 hasta 5, tomando valores iniciales en 0 e iterando las variables de lectura recibidas por la cantidad de estrellas.

```
class cantidadEstrellas{
  mostrar(registros){
    const counts = [0, 0, 0, 0, 0];
    registros.forEach(r => counts[r.estrellas - 1]++);
    console.log('\n Cantidad por Calificación:');
    counts.forEach((c, i) => {
    console.log(`${i + 1} estrella(s): ${c}`);
  });
  }
}
```

Salir:

La función salir es nuestra opción 8 y servirá para dar por terminado el programa, en caso de elegir una opción invalidad, el mismo programa marca un error donde se pide elegir una opción de las ocho dentro del menú.

```
        case '8':
          console.log('Saliendo...');
          process.exit(1);
        default:
          console.log(' !Error: elige una de las 8 opciones');
          this.menu();
```

Diagrama de flujo:

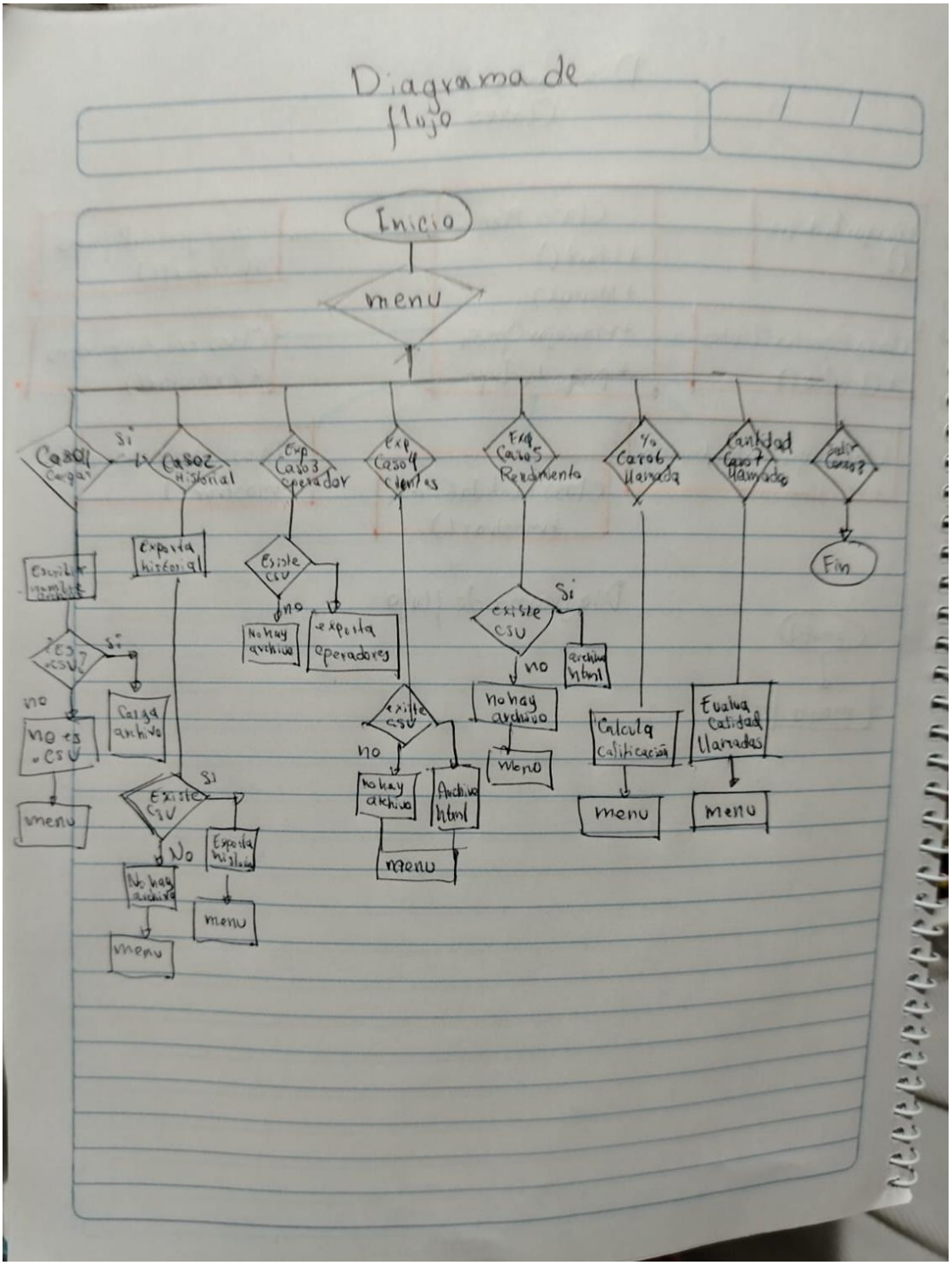
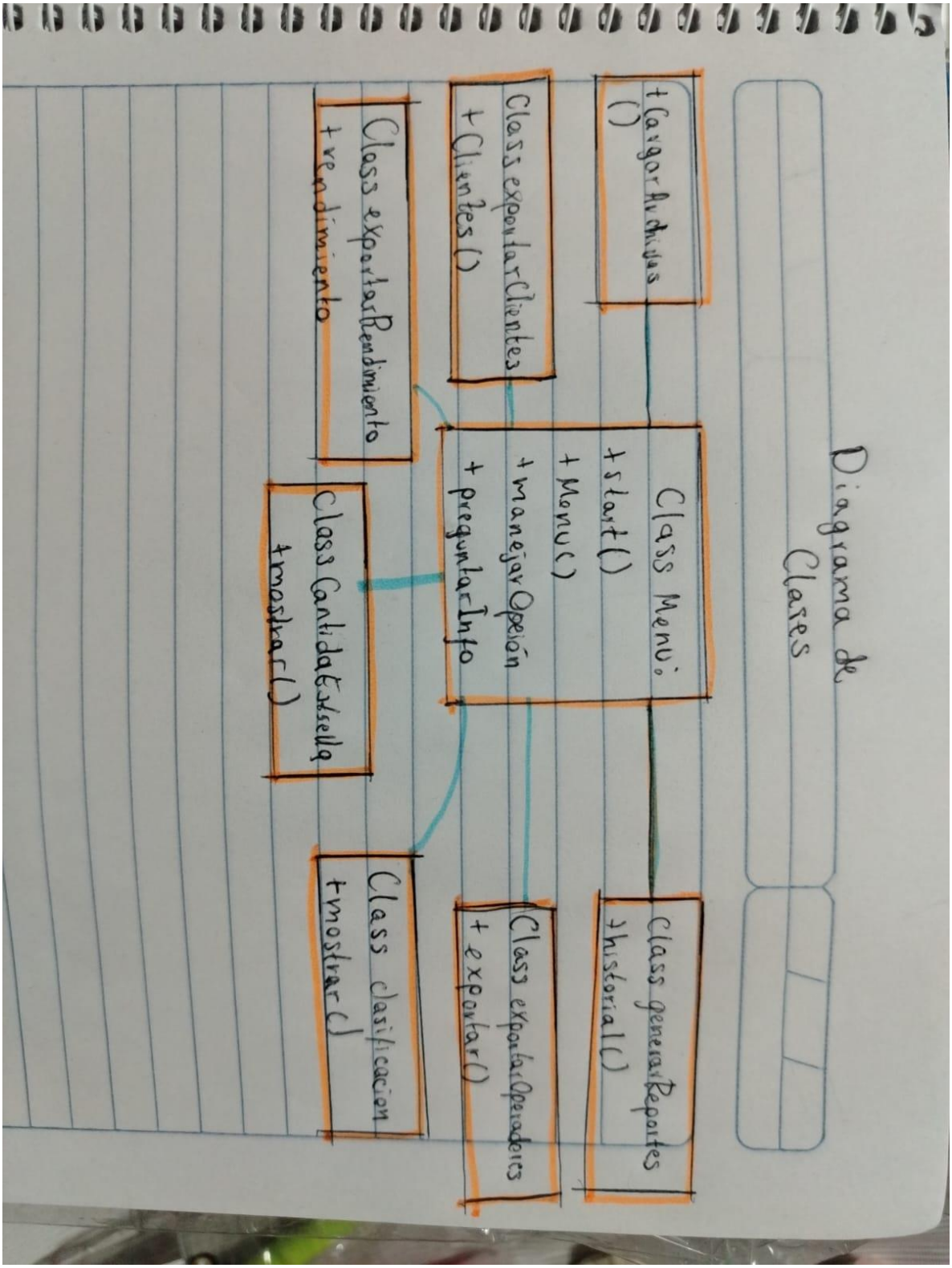
Diagrama de clases: